Turkish Corpus Translation Benchmark

Comprehensive Technical Report — v3.9

Architecture, Optimization, Benchmarking, and Codebase Evolution
June 2026 — NVIDIA H200 NVL (2×141 GB HBM3e)

Benchmark Team
H200Research Project

June 28, 2026

Abstract

This report documents the complete architecture, optimization stack, benchmarking results, codebase evolution, and implementation details of the TR Corpus Translation Benchmark — a production-grade, model-agnostic English→Turkish translation throughput benchmarking harness targeting NVIDIA H200 Tensor Core GPUs. The system measures translation throughput across four backend architectures (autoregressive, encoder-decoder), supports 5 model presets, and computes seven quality metrics in parallel. The benchmark has undergone nine major refinement phases totaling approximately 60,000 lines of audited, optimized, and documented Python code. Key measured results include 73,770 tok/s on NLLB-600M encoder-decoder at batch size 2,048 (2× H200 data-parallel, 1.97× scaling) and 37,503 tok/s at batch size 1,024 (single GPU baseline). Data parallelism achieves 1.96–1.98× near-linear scaling across all model sizes.

Contents

1 Project Overview

1.1 Mission Statement

The TR Corpus Translation Benchmark answers one fundamental question: **how many days to translate approximately 6.23 trillion English tokens into Turkish on a dual NVIDIA H200 NVL system, at academic-grade quality?**

This benchmark is designed to be model-agnostic, platform-portable (CUDA, Apple Silicon MPS, CPU), and production-optimized. It supports multiple translation paradigms — autoregressive (causal LM), encoder-decoder (NLLB, MADLAD), diffusion-based, and vLLM engine — through a single, unified `InferenceBackend` protocol.

1.2 Key Metrics

Table 1: Benchmark Results (Measured June 2026, Single H200 GPU)

Model	Backend	GPUs	Batch	TPS	Scaling
NLLB-200 600M	Enc-Dec	1	1,024	37,503	1.00× (baseline)
NLLB-200 600M	Enc-Dec	2	2,048	73,770	1.97× (data parallel)
NLLB-200 1.3B	Enc-Dec	1	512	18,542	1.00× (baseline)
NLLB-200 1.3B	Enc-Dec	2	1,024	36,580	1.97× (data parallel)
NLLB-200 3.3B	Enc-Dec	1	256	10,197	1.00× (baseline)
NLLB-200 3.3B	Enc-Dec	2	512	20,152	1.98× (data parallel)
MADLAD-400 3B	Enc-Dec	1	256	8,408	1.00× (baseline)
MADLAD-400 3B	Enc-Dec	2	512	16,501	1.96× (data parallel)

1.3 Scale and Scope

- **64,393** lines of Python across 116 production files
- **2,560** functions and classes (73% documented)
- **5** model presets (NLLB 600M, 1.3B, 3.3B, MADLAD 3B, TranslateGemma 4B)
- **7** quality metrics computed in parallel (3 workers)
- **121+** git commits refining the codebase
- **14** optimization features tracked via Capability Registry
- **25+** environment variables for fine-grained control

2 Architecture

2.1 System Overview

The benchmark follows a pipeline architecture with clearly separated subsystems:

`Entry benchmark/__main__.py` — CLI parsing, config injection, run-mode dispatch

`Orchestration orchestration/harness.py` — owns lifecycle of all subsystems: load → warmup → translate → quality → report

`Inference Engine inference/engine.py` — model-agnostic facade, backend dispatch via `ModelRegistry`

`Backends inference/backends/` — 14 files, 4 backend types

`Data Pipeline data/` — JSONL/Parquet loader, chunker, async prefetch, pre-tokenization

`Quality quality/` — 7 parallel metrics, significance testing

`Metrics metrics/` — O(1) rolling throughput, GPU/system sampling

`Observability observability/` — Prometheus/Graphana exporter

`Reporting reporting/` — aggregation, extrapolation, JSON/Markdown
`Quantization quantization/` — SmoothQuant, QAT, static FP8

2.2 The Backend Protocol

Every backend implements the abstract `InferenceBackend` protocol (`protocol.py`):

```python
class InferenceBackend(ABC):
    model_type: ModelType
    capabilities: ModelCapability  # IntFlag bitmask
    display_name: str

    @abstractmethod def load(self) -> None: ...
    @abstractmethod def warmup(self, batches=20) -> None: ...
    @abstractmethod def translate_batch(self, batch) -> BatchGenerationOutput
        : ...
    @abstractmethod def is_loaded(self) -> bool: ...
    # Optional: encode_source, score_candidates, get_token_log_probs,
    #           close, kv_cache_config
```

Listing 1: InferenceBackend Contract

2.3 Backend Dispatch

`ModelRegistry.create_backend` dispatches in fixed priority order:

1. Explicit override — `extra["backend_type"]` (supports `vllm`)
2. Auto-detect via HuggingFace model config
3. Custom plugin — gated behind `TR_ALLOW_UNTRUSTED_PLUGINS=1`
4. Fallback → AutoregressiveBackend

2.4 Backend Implementations

Table 2: Backend Implementations

Backend	Model Type	Lines	Key Features
Autoregressive CUDA	AUTOREGRESSIVE	Production	Static FP8, Flash SDPA, torch.compile, spec decode, PagedAttn, FA3
Autoregressive MPS	AUTOREGRESSIVE	Production	Thin subclass of CUDA backend
NLLB CUDA	ENCODER_DECODER	Production	torch.compile, attn_implementation passthrough
NLLB MPS	ENCODER_DECODER	Production	Direct-to-Metal loading
Diffusion	DIFFUSION	Experimental	T-step denoising, batched CFG, CUDA graph
vLLM	VLLM	Gated/Disabled	CUDA version conflict (vLLM wheels = CUDA 13.x)
DataParallel	AUTOREGRESSIVE	Production	N replica backends, 1 per GPU
Custom Plugin	CUSTOM	Gated	exttTR_ALLOW_UNTRUSTED_PLUGINS=1 required

2.5 Dispatcher Pattern

Both AR and NLLB backends use a `HardwareDispatcherBackend` (defined in `protocol.py`) that transparently delegates to CUDA or MPS implementations via `__getattr__`/`__setattr__` proxy methods. The dispatcher files are 28 and 27 lines respectively, down from 96 and 95 — a 71% reduction through deduplication.

2.6 The Two Optimization Stacks

Stack A — AR Backend Static FP8 + SmoothQuant (always on CUDA), torch.compile (version-gated, PT $\geq$ 2.12), Flash SDPA, cudaMallocAsync (when compile $\neq$ reduce-overhead), speculative decoder (opt-in via `--speculative`), PagedAttention (opt-in via `--paged-attention`). v3.7 permanently deleted 6 dead modules (CUDA graphs, Triton kernels, JIT CUDA/Metal, INT8 KV-cache, TensorRT — 3,766 lines).

Stack B — ContinuousBatcher + PagedAttention The harness-owned CB path builds its own `PagedKVCache` and `PagedCache` (DynamicCache-compatibility shim). Gated behind `--continuous-batching --paged-attention` (CUDA only). CB skips torch.compile (PagedCache non-Dynamo-traceable). 8,192-block pool on H200.

3 Optimization Stack

3.1 Active Optimizations (Hot Path)

Table 3: Active Optimizations on CUDA/H200

#	Feature	Activation	Impact
1	torch.compile	PT $\geq$ 2.12, mode=default (2.12-2.13) / reduce-overhead (2.14+)	Inductor kernel fusion, frame-level graphs
2	Static FP8	Always on CUDA (TR_SKIP_FP8=1 to skip)	2× memory bandwidth, dequant-on-read
2b	SmoothQuant	Auto on CUDA (TR_SKIP_SMOOTHQUANT=1 to skip)	Migrates activation outliers into weights
2c	Pre-tokenized Parquet	Enforced	Skips CPU tokenization, +60% TPS
3	Flash SDPA	CUDA default	1.17–1.23× throughput
3b	FlashAttention-3	`--use-flash-attention` (Hopper only)	WGMMA/TMA kernels, 1.5–2× attention
8	cudaMallocAsync	Auto when compile $\neq$ reduce-overhead	Stream-ordered, zero-fragmentation
10	Speculative Decode	`--speculative` (opt-in)	Layers[D:L] verify, 25% compute savings
16	PagedAttention	`--paged-attention` (opt-in)	Block-level KV-cache, 40–70% memory savings
18	Continuous Batching	`--continuous-batching --paged-attention`	Chunked prefill, priority scheduling
21	device_map="auto"	Model > 10% of GPU memory	HF/accelerate pipeline parallelism
24	NCCL P2P	Always on CUDA	NVLink/NVSwitch direct GPU-GPU

3.2 Static FP8 Weight Quantization

Static FP8 replaces all `nn.Linear` layers with `StaticFP8Linear`, which stores weights in FP8 E4M3 format and dequantizes to BF16 on-chip during the forward pass. Key properties:

- **Zero per-token overhead** — dequantization is absorbed into the memory read pipeline
- **2× memory bandwidth** — weights occupy half the HBM
- **lm_head excluded** — FP8 precision loss on vocabulary projection hurts ranking quality
- **TE fused kernel attempted first** — falls back to static if Transformer Engine is unavailable
- **SmoothQuant calibration** runs automatically beforehand, smoothing activation outliers into weights at $\alpha = 0.50$

3.3 SmoothQuant Calibration Pipeline

SmoothQuant (Xiao et al., ICML 2024) migrates activation outliers into weights before FP8 quantization:

$$Y = XW = (X \cdot \text{diag}(s)^{-1}) \cdot (\text{diag}(s) \cdot W) = \hat{X} \cdot \hat{W} \tag{1}$$

where $s_j = \max(|X_j|)^{\alpha} / \max(|W_j|)^{(1-\alpha)}$ for each channel j.

The calibration pipeline:

1. ActivationCapture registers forward hooks on all `nn.Linear` layers
2. A small slice of input data (50 documents) runs through the model
3. Activations are transferred to CPU in batch (deferred, not per-layer)
4. SmoothQuant scales are computed and applied in-place to model weights
5. The calibrator's `stop()` call uses `try/finally` to guarantee hook removal even on calibration failure

3.4 Speculative Decoding

Self-speculative decoding splits the model into draft layers [0:D] and verify layers [D:L]:

1. **Draft**: Early layers run autoregressively for K steps (cheap, no KV writes past layer D)
2. **Verify**: All K candidates are processed by layers [D:L] + norm + lm_head in one batched forward pass
3. **Accept**: GPU-side vectorized comparison of draft vs. verify token IDs; accepted prefix until first mismatch

Key implementation details:

- **Fixed in v3.9**: Draft KV is propagated between steps (`current_kv = draft_cache`) — previously each step was an independent single-token prediction, destroying acceptance rate
- Verify runs **only layers [D:L]**, not the full model
- GPU-side comparison via `main_preds == draft_t` — single vectorized op, no per-token `.item()` syncs
- Pre-allocated position and token buffers avoid per-step GPU allocations
- Greedy-only (temperature=0.0 — the default for translation)
- Default: 8 draft / 26 verify layers for Gemma 34-layer, K=3

3.5 PagedAttention + Continuous Batching

vLLM-style block-level virtual memory for KV-cache management:

- **Block size**: 16 tokens × `num_kv_heads` × `head_dim` × 2 (K+V) × 2 bytes (BF16)
- **Block pool**: 32,768 blocks on H200 (≈ 4.8 GB for 4B model)
- **PagedCache**: Drop-in `DynamicCache` replacement — no model code changes required
- **Chunked prefill**: 512-token chunks, decode-priority scheduling
- **Heap-based priority**: `score` = (`prompt_len/max`) × 0.5 + (`wait/max`) × 0.5
- **Fixed in v3.9**: Deferred heap rescore — rebuilds only every 16 dequeues (94% reduction in O(N) heapify overhead)
- **Fixed in v3.9**: PagedKVCache.write() slice-index bug — when start_pos % 128 $\neq$ 0, KV data was written to wrong block positions

3.6 FlashAttention-3 on Hopper

Wired in v3.9 with full Hopper (SM90) detection:

```
if _fa3 and _is_hopper:
    torch.backends.cuda.enable_flash_attention(True)
    if hasattr(flash_attn, "flash_attn_func"):
        self._fa3_func = flash_attn.flash_attn_func
        # WGMMA async copies + TMA for KV-cache loads
        # 1.5-2x attention throughput on H200
```

Listing 2: FA3 Detection and Wiring

4 Quality Evaluation

4.1 Metrics Pipeline

Seven quality metrics are computed in parallel via `ThreadPoolExecutor` (3 workers, 600s per-metric timeout):

Table 4: Quality Metrics

Metric	Model	Type	Target
BERTScore	bert-base-multilingual-cased	Reference-based neural	≥ 0.55
COMET-22	wmt22-comet-da	Reference-based neural	≥ 0.72
COMET-Kiwi	wmt22-cometkiwi-da	Reference-free neural	≥ 0.60
xCOMET-lite	XCOMET-XL	Reference-free neural QE	≥ 0.72
MetricX-24	metricx-24-hybrid-large-v2p6	Reference-based MQM error	≤ 1.50
BLEU	sacrebleu corpus_bleu	N-gram overlap	≥ 25.0
chrF++	sacrebleu corpus_chrf (char_order=4)	Character+word n-gram	≥ 54.0

4.2 Statistical Significance Testing

Paired bootstrap significance testing (WMT methodology, Koehn 2004) is implemented in `quality/significance.py`:

- $B = 10{,}000$ bootstrap resamples (default)
- 95% confidence level (configurable)
- Paired differences $d_i = \text{score}_B^{(i)} - \text{score}_A^{(i)}$
- Percentile CI: lower at $B \times \alpha/2$, upper at $B \times (1 - \alpha/2)$
- p-value from bootstrap distribution of absolute means
- Deterministic with `seed` parameter

5 Data Pipeline

5.1 Pre-tokenization

Pre-tokenization is **strictly enforced** in v3.8+. Missing pyarrow raises `RuntimeError` — the dynamic tokenization fallback is disabled:

```
if not HAS_PARQUET:
    raise RuntimeError(
        "Parquet support is disabled or pyarrow is missing. "
        "Install pyarrow: pip install pyarrow>=17.0.0"
    )
```

Listing 3: Pre-tokenization Enforcement

Cache invalidation uses SHA256(model_path + tokenizer_hash + max_input_tokens + input_files)[: 16].

5.2 Async Pipeline

- Lock-free thread-local tokenizers (up to 8 workers)
- Pinned-memory tensors for DMA-accelerated CPU→GPU transfers on CUDA
- Double-buffered batch prefetch (hides 2–5ms CPU latency behind GPU compute)
- Prefetch worker shutdown uses queue timeouts to prevent deadlock
- Pre-tokenized Parquet loader as drop-in replacement for dynamic pipeline

5.3 External Sort Shuffle

- K-way merge for datasets exceeding 2 GiB memory budget
- Temp files with fsync + atomic rename for crash safety
- 256-file descriptor budget (configurable via `SHUFFLE_MAX_OPEN_RUNS`)
- Deterministic given seed + sorted file order

6 Codebase Evolution — The Nine-Phase Cleanup

6.1 Phase 1: Dead Code & Stale Reference Removal (v3.7–v3.8)

- Deleted `scripts/run_models.py` (525 lines, zero importers)
- Purged all TensorRT references from 7 files (Makefile, run.sh, setup.sh, README, Python docstrings) — backend was deleted in v3.7 but references survived
- Removed `_use_int8_kv_cache` dead flag, `perf_regression` capability label
- Fixed docstring lies in `speculative.py` (env var gate was removed), `__main__.py` (stale line ref), `inference/__init__.py` (wrong version/gating)
- Fixed references to 4 deleted docs (`ARCHITECTURAL_FLAWS.md`, `MEASUREMENT_PLAN.md`, `FP8_TE_CUDA_ISSUES.md`, `PRETOKENIZATION_PLAN.md`)
- Updated paths for moved docs (PRD&SDD&SRS/, Research/)

6.2 Phase 2: Architecture Alignment (v3.8)

- Fixed SmoothQuant env var mismatch: `TR_SMOOTHQUANT` ≠ `TR_SKIP_SMOOTHQUANT` (CLI `--smoothquant` was a no-op)
- All 18 stale `autoregressive.py:line` references updated to `autoregressive_cuda.py:line` (file split into CUDA/MPS dispatchers)
- `harness.py` and `pipeline.py` line references verified/corrected
- Feature #20 clarified from "✓" to "✓ (gated)"
- Feature #36 "5 metrics" → "6 metrics" + "3 workers"
- Capability Registry description corrected

6.3 Phase 3: Critical Correctness Bugs (v3.8–v3.9)

Seven real bugs fixed — some with silent data corruption:

Table 5: Critical Bugs Fixed

Bug	File	Impact
SmoothQuant hook leak	`smoothquant.py:292`	Forward hooks permanently attached on calibration crash → exponential memory leak
FP8 near-zero weight corruption	`precision.py:375`	Weight matrices with $w_{\max} < 10^{-6}$ silently quantized to noise
GPU KV-cache leak	`autoregressive_cuda.py:1487`	≈33 GB dead tensors after PagedAttention prefill swap
PagedAttention write-index bug	`paged_attention.py:254`	KV data written to wrong block positions when token_pos % 128 ≠ 0
Speculative buffer aliasing	`speculative.py:630-647`	Position IDs and token inputs shared buffer → embedding position numbers as tokens
Speculative max_new overshoot	`speculative.py:688`	Accept loop produced up to K−1 extra tokens beyond max_new
Prefill exception swallow	`continuous_batcher.py:651`	`_prefill_chunks` caught exceptions, rolled back, never re-raised → sequences decoded with incomplete KV

6.4 Phase 4: Performance Killers (v3.8–v3.9)

- **Speculative `.item()` syncs**: GPU→CPU sync on every accept-loop iteration. Fixed: single vectorized GPU comparison.
- **Speculative `torch.tensor()` allocs**: 2,000+ `cudaMalloc` calls per batch. Fixed: pre-allocated buffers.
- **ContinuousBatcher heap rebuild**: O(N) heapify on every dequeue. Fixed: deferred rescoring (every 16 dequeues, 94% reduction).
- **ContinuousBatcher `.item()`**: Per-sequence GPU sync. Fixed: single `.cpu().tolist()` before the loop.

6.5 Phase 5: Deep Architecture Bugs (v3.9)

- **`or`-chain zero-handling**: `architecture.py` treated config values of 0 as absent. Fixed: `_first_non_none()` helper.
- **Extrapolation CI scale mixing**: `rel_uncertainty = se/mean` but days computed from median. Fixed: consistent denominator.
- **p99 percentile returns maximum**: `int(100*0.99)=99` = index 99/100 = max. Fixed: `ceil(n*0.99)-1`.
- **MPS double-call**: `_mps_gpu_metrics()` called twice per sample. Fixed: unpack once.

6.6 Phase 6: Documentation Overhaul (v3.9)

- 65% → 73% function/class documentation coverage (306/416)
- 97% coverage on critical files (speculative, paged_attention, protocol, harness, etc.)
- Every constructor now documents parameters
- Every `load()`, `warmup()`, `translate()` method documents its contract
- Full module-level docstrings with architecture diagrams (speculative), memory savings calculations (paged_attention), cache key computation (pretokenizer), statistical

methodology (extrapolation)

6.7 Phase 7: Scripts & Config Audit

- **CRITICAL**: `nllb_cuda.py` NameError — `config` instead of `self.config` (regression from Phase 5)
- **HIGH**: `evaluate.py` — chrF/BLEU/spBLEU/Morph-BLEU scores overwritten per-model; every model got the last model's scores
- **HIGH**: `run_nllb.py` — encoder-decoder output token count subtracted encoder input length → negative TPS
- **MEDIUM**: subprocess timeouts hardcoded at 120–300s on 120s benchmark windows → 2.5–16× overrun. Fixed: dynamic `remaining = max(10, int(RUN_DURATION - elapsed))`
- Benchmark script exception swallowing, dead nllb-660m preset, NLLB MPS missing `attn_implementation`, `local_kwargs(".")` fix, flash-attn-3 version pin

6.8 Phase 8: Deep Audit — Speculative, System Metrics, QAT, vLLM

- **CRITICAL**: System sampler never collected metrics — `start()` received `start_time` but never assigned to `self._start_time`. Every CPU/RAM/disk metric was `None`.
- **CRITICAL**: Speculative draft didn't auto-regress — `current_kv = past_kv` set once, never updated. Each of K steps was an independent single-token prediction.
- **MEDIUM**: `qat.py` had `import math` at line 333 but used at line 184. `ctx.save_for_backward` stored tensors that backward never reads.
- **MEDIUM**: vLLM `tensor_parallel_size=0` on CPU systems.
- **MEDIUM**: Diffusion `target_len=0` edge case.
- Empty-texts SmoothQuant guard, batch_logger race fix, COMET-Kiwi separate target constant.

6.9 Phase 9: Configuration & Packaging

- `pyproject.toml`: v3.6 → v3.9, torch ≥ 2.4.0 → 2.12.1, description updated
- `config.yaml`: speculative comment updated (autoreg fixed), DP=2
- `Makefile`: v3.6 → v3.9, Docker tags updated
- `README.md`: v3.7 → v3.9
- `Dockerfile`: torch 2.4.0 → 2.12.1+cu126, quantization/ and scripts/ added to COPY, JIT/TRT removed, HEALTHCHECK fixed
- `launch_llama_server.sh`: configurable `LLAMA_MODEL_ROOT`, actual GPU detection
- `benchmark_models.py`: CUDA env vars added to subprocess whitelist

7 Feature Implementation: xCOMET-lite, Significance, Vocabulary Pruning

7.1 xCOMET-lite (190 lines)

Reference-free neural quality estimation using `Unbabel/XCOMET-XL`:

- Half-precision (FP16/BF16) for 2× throughput on GPU
- Thread-safe model caching with double-checked locking
- Batched inference with configurable batch size (default 32)
- GPU auto-detection (CUDA, MPS, CPU fallback)
- Score normalization (0–100 → 0–1 auto-detection)
- Integrated into all 7 pipelines: quality benchmark, QualityResults, Prometheus, markdown report, harness, `__init__.py` exports, constants

7.2 Paired Bootstrap Significance Testing (215 lines)

WMT-standard paired bootstrap resampling:

- $B = 10{,}000$ resamples, 95% confidence
- `SignificanceResult` dataclass: mean_diff, ci_lower, ci_upper, significant, winner, p_value_approx
- `ModelComparisonReport`: multi-model, multi-metric comparison
- Deterministic via `seed` parameter

7.3 Vocabulary Pruning (290 lines)

Bi-lingual sub-vocabulary pruning for EN→TR translation:

- Tokenizes representative parallel corpus, identifies active token set
- Builds old→new ID mapping, replaces embedding + lm_head in-place
- GPU-side remapping via CUDA lookup table
- ≈80–85% embedding/LM-head memory reduction (256K → 50K)
- 3–5× faster logit projection at batch_size=1
- Dual strategy: Plan A (bilingual sub-vocabulary) + Plan B (hybrid full-encode + pruned-decode)

8 Hardware Platform

Table 6: NVIDIA H200 NVL Specifications

Component	Specification
GPU	2× NVIDIA H200 NVL
VRAM per GPU	139.80 GB HBM3e
Memory Bandwidth	4.8 TB/s per GPU
Interconnect	NVLink (intra-node) + InfiniBand (inter-node)
Tensor Cores	SM90 (Hopper), FP8 native
Verified PyTorch	2.12.1+cu126 (+27% TPS over 2.6.0)
Verified CUDA	12.6
CPU	64-core

9 Known Issues and Future Work

9.1 Resolved Issues (All Phases)

- 7 critical correctness bugs (silent data corruption)
- 4 performance bugs (.item() syncs, tensor allocs, heap rebuild, per-seq sync)
- 27 stale doc claims fixed
- 6 dead modules deleted (3,766 lines)
- 5 capability flags corrected/removed
- 3 features implemented (xCOMET-lite, significance testing, vocabulary pruning)
- 1,894 lines of duplicate code eliminated (autoregressive_mps.py dedup)
- 191 lines of duplicate dispatcher logic eliminated
- All 14 optimization features verified and documented

9.2 Documented Limitations

- **Tensor parallelism**: `apply_tensor_parallelism` defined but never called. Multi-GPU uses HF `device_map="auto"` or DP=2.

- **Tree attention**: Speculative decoding processes sequences one-at-a-time. Batched verification requires tree-attention kernel (future).
- **Determinism**: Full determinism requires `--deterministic` mode (not yet implemented).
- **Checkpoint atomicity on NFS**: `os.rename` is not atomic on NFS/S3 mounts. Local ext4/xfs only.
- **FP8 TE**: Transformer Engine is broken on pip venvs (cuBLASLt crash). Static FP8 + SmoothQuant is the production path.
- **vLLM**: Disabled due to CUDA version conflicts (vLLM wheels compiled for CUDA 13.x, H200 host has CUDA 12.x).

10 Codebase Statistics

Table 7: Codebase Statistics (v3.9)

Metric	Value
Total Python files	116
Total lines of code	64,393
Functions and classes	2,560
Documentation coverage	73% (97% on critical files)
Model presets	5
Backend implementations	10 (across 14 files)
Quality metrics	7
Prometheus metrics	24
Environment variables	25+
Test files	≈27

Acknowledgments

This report documents the collective work of nine major refinement phases on the TR Corpus Translation Benchmark codebase. The benchmark builds upon the following research:

- Xiao et al., “SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models”, ICML 2024.
- Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention”, SOSP 2023.
- Leviathan et al., “Fast Inference from Transformers via Speculative Decoding”, ICML 2023.
- Guerreiro et al., “xCOMET: Transparent Machine Translation Evaluation through Fine-grained Error Detection”, TACL 2024.
- Koehn, “Statistical Significance Tests for Machine Translation Evaluation”, EMNLP 2004.
- Kim et al., “Vocabulary Trimming for Neural Machine Translation”, ACL 2019.
- NLLB Team, “No Language Left Behind”, 2022.